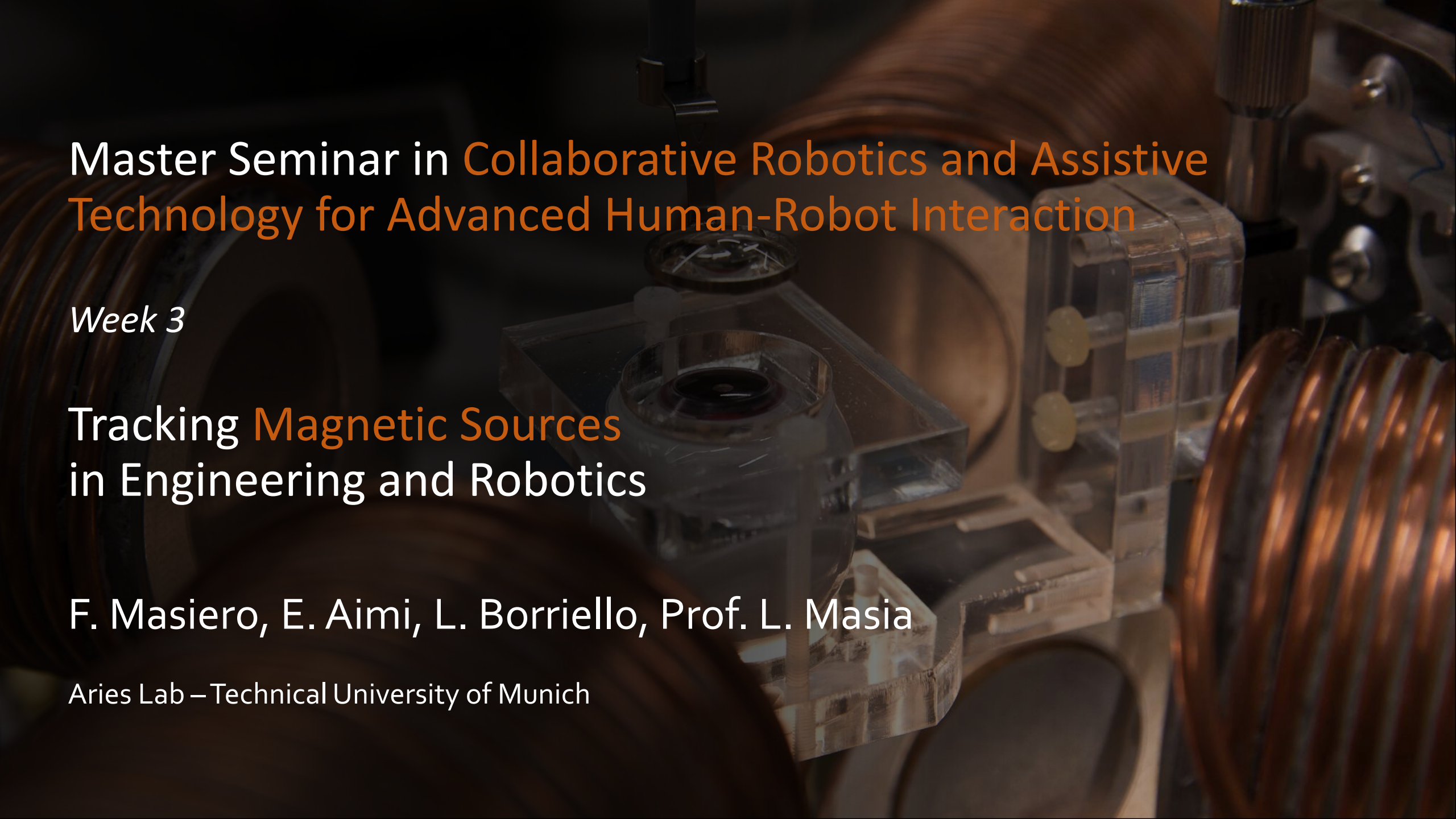




# Master Seminar in Collaborative Robotics and Assistive Technology for Advanced Human-Robot Interaction

*Week 3*

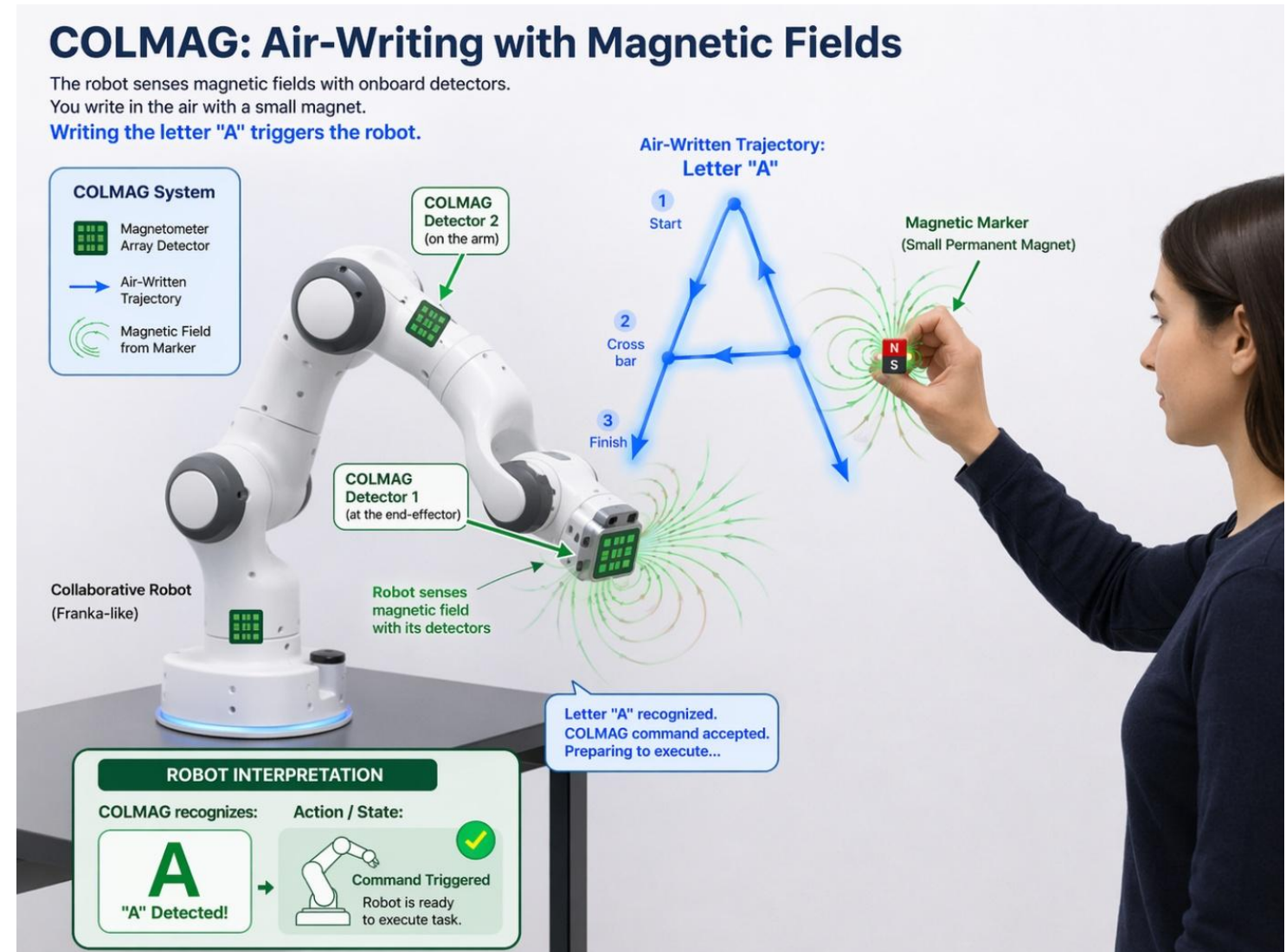
## Tracking Magnetic Sources in Engineering and Robotics

F. Masiero, E. Aimi, L. Borriello, Prof. L. Masia

Aries Lab – Technical University of Munich

# Projectual Work

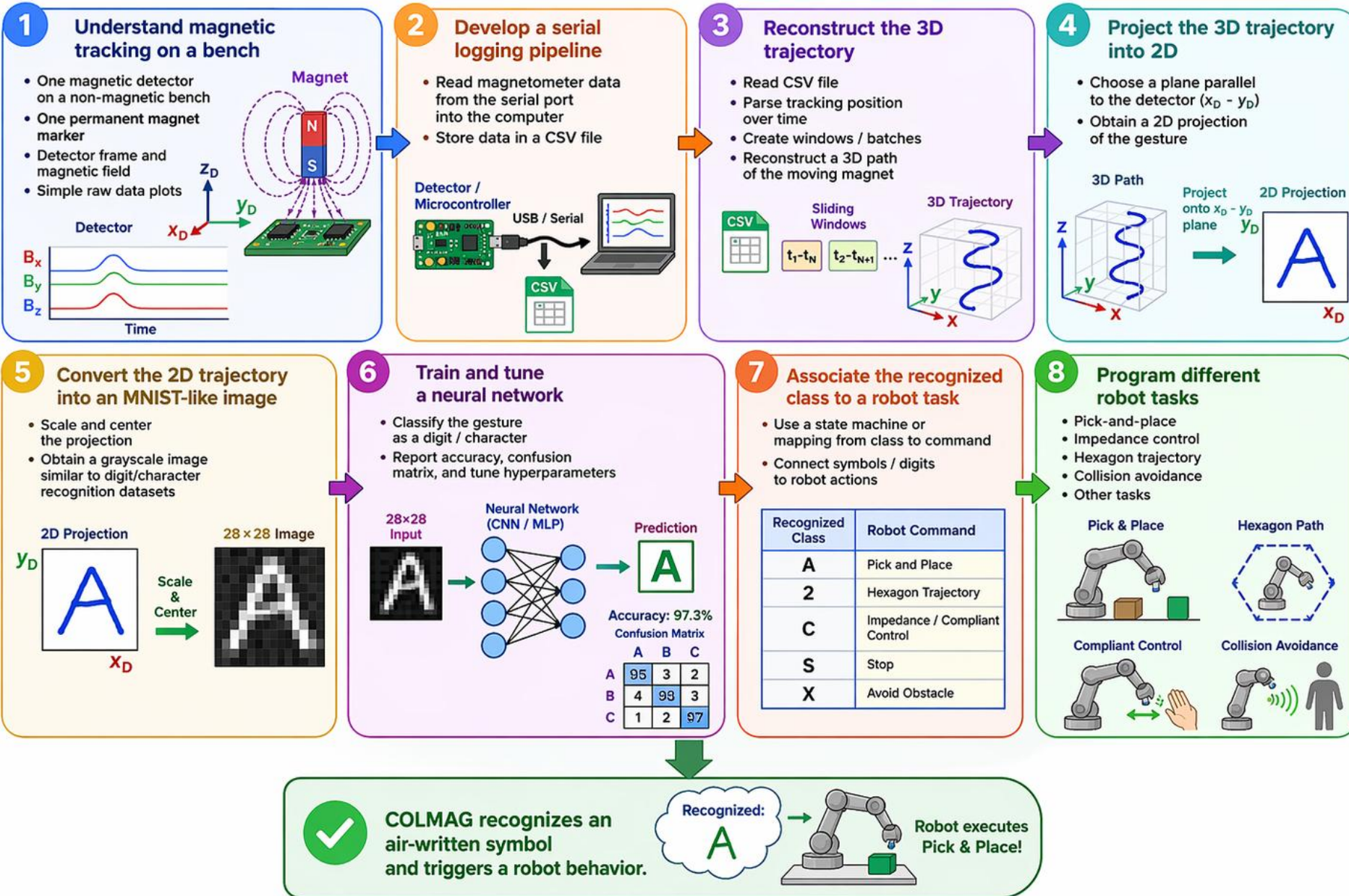
Research project with an 8-week plan





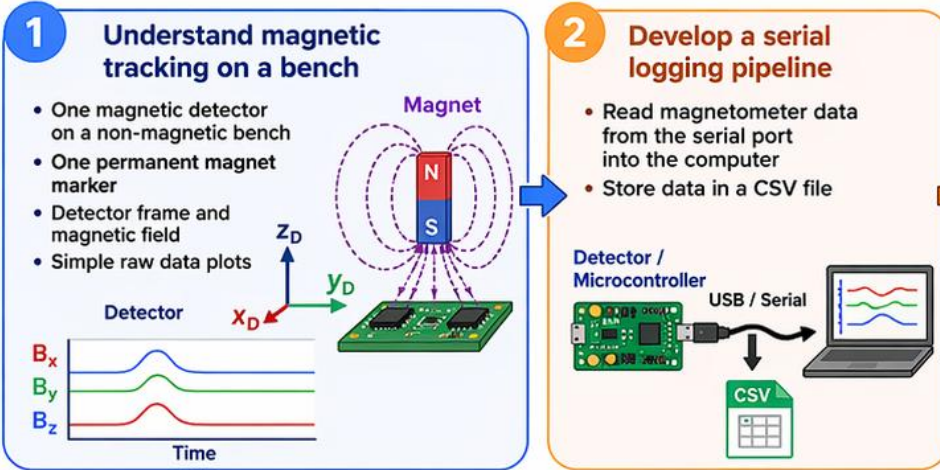
# COLMAG Work Plan

From magnetic sensing to robot actions



# COLMAG Work Plan

*From magnetic sensing to robot actions*



*In the last episode of this Seminar ...*

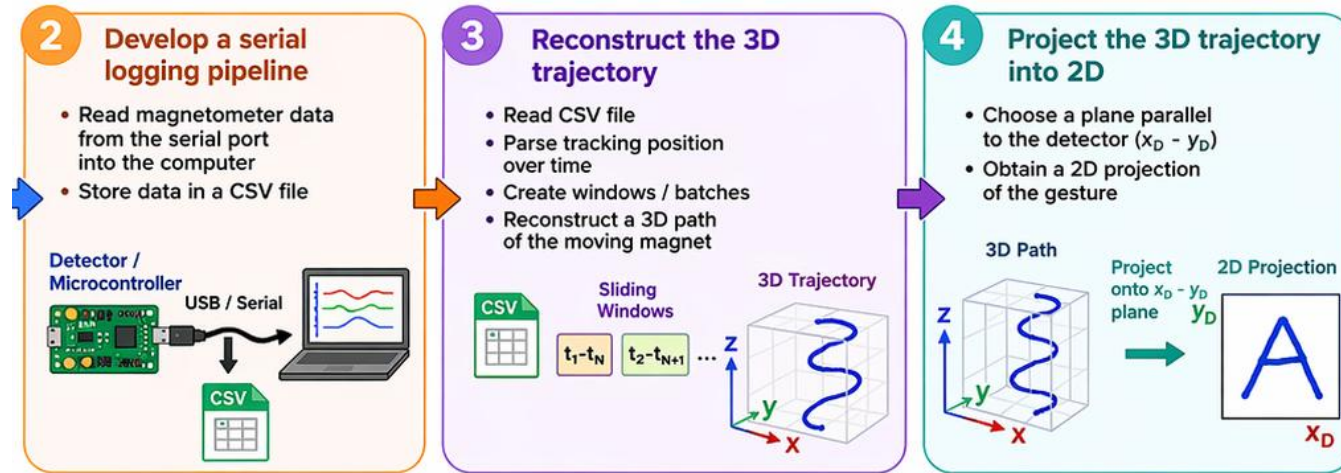
*What you should have done, or what you should try to get by the end of the day*

- *Reading the data stream of a 16-sensor magnetic detector*
- *Plotting data in real-time for immediate visualization*
- *Storing data in a CSV file to get some data to play without the detector*

# COLMAG Work Plan

From magnetic sensing to robot actions

*In this episode of the Seminar ...*



**GOAL:** Produce the pipeline to get 2D representations of air-written digits from 3D magnet position data. Extend the approach by building the logic to recognize when the user is pressing virtual switches.



# Projectual Work – Warmup Start

*Magnetic Detector*

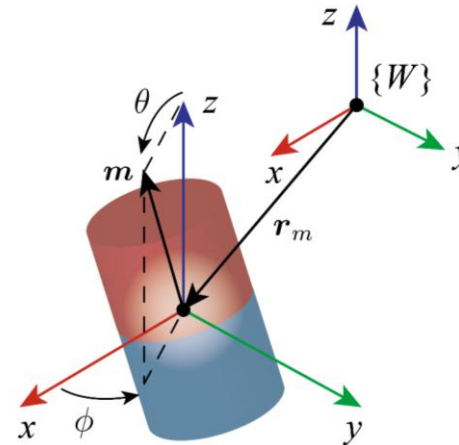
Magnet

USB connection  
Serial Communication

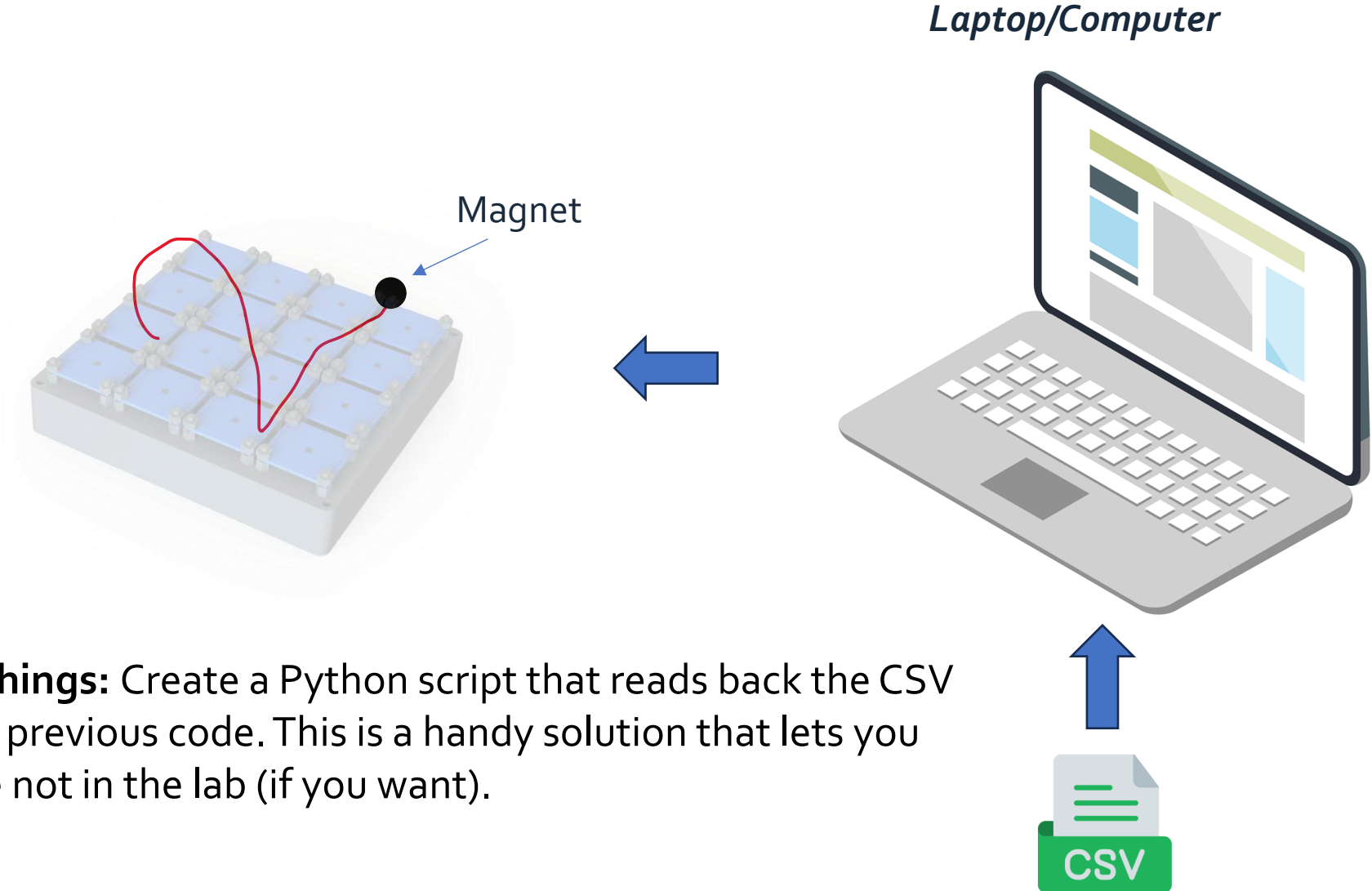
[0xAA, Bx1, By1, Bz1, ..., Bx16, By16, Bz16, x, y, z, mx, my, mz, 0xBB]

*Laptop/Computer*

$\min \left( \vec{B}_s - \vec{B}_c \right)$   
Magnet Tracking Pipeline



# Projectual Work – Warmup Start



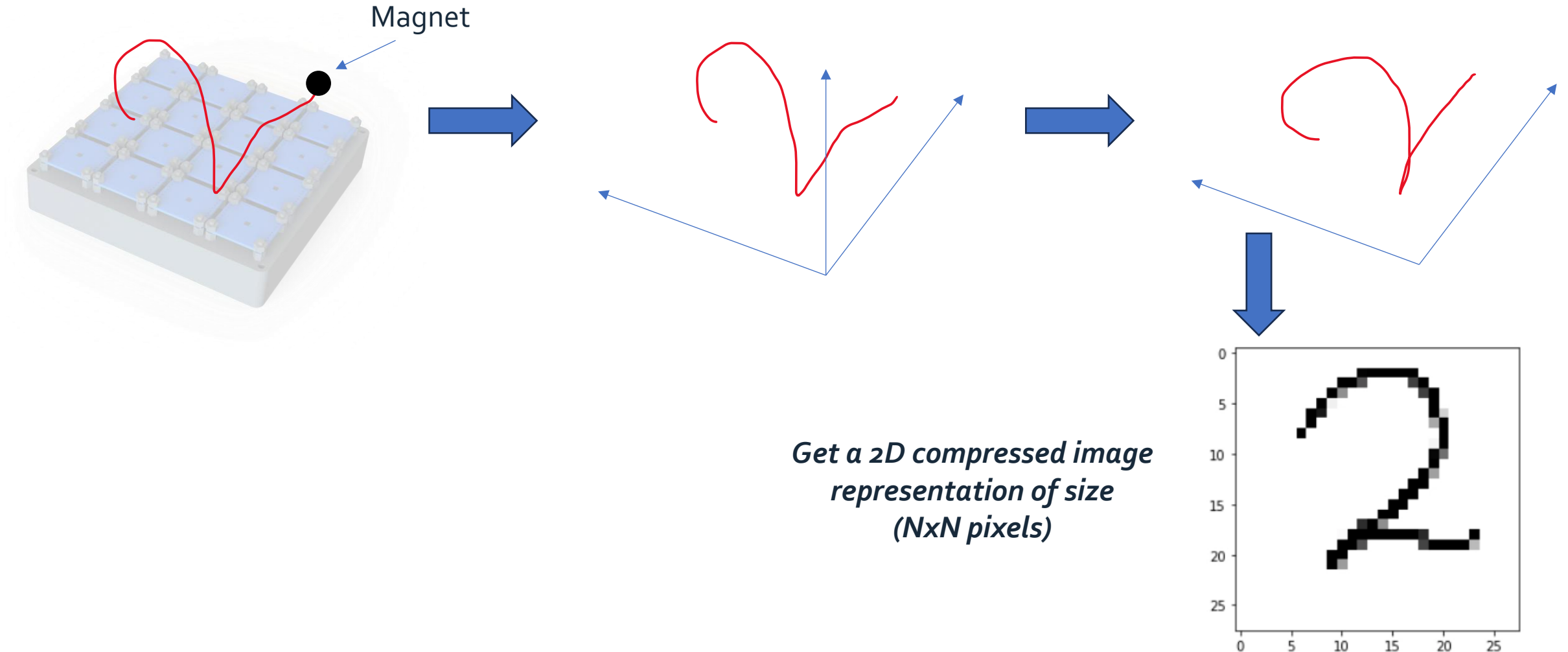
**Before starting with the cool things:** Create a Python script that reads back the CSV file that you recorded with your previous code. This is a handy solution that lets you check things even when you are not in the lab (if you want).

# Projectual Work – Warmup Start

*Real 3D Trajectory*

*3D representation in Python*

*2D representation in Python*



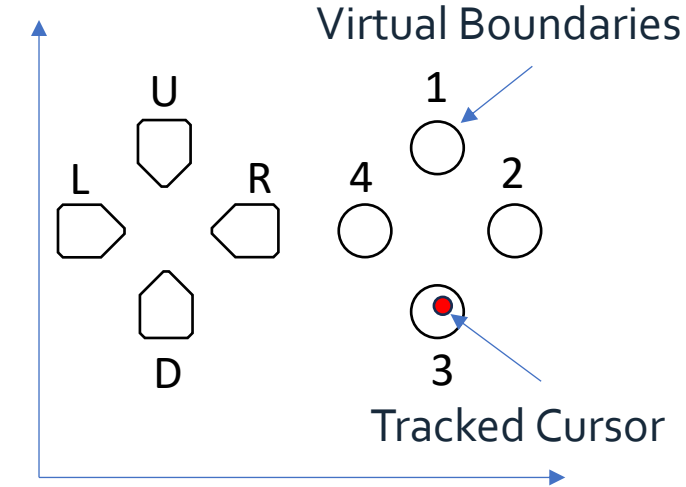
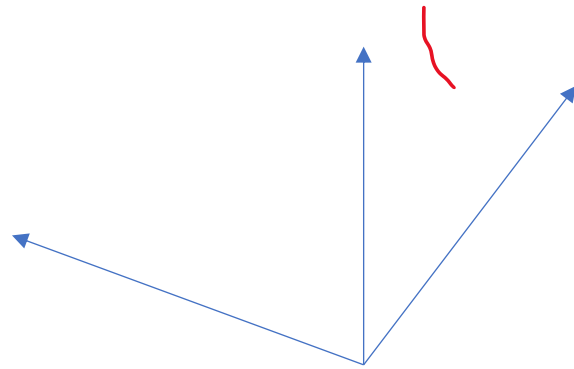
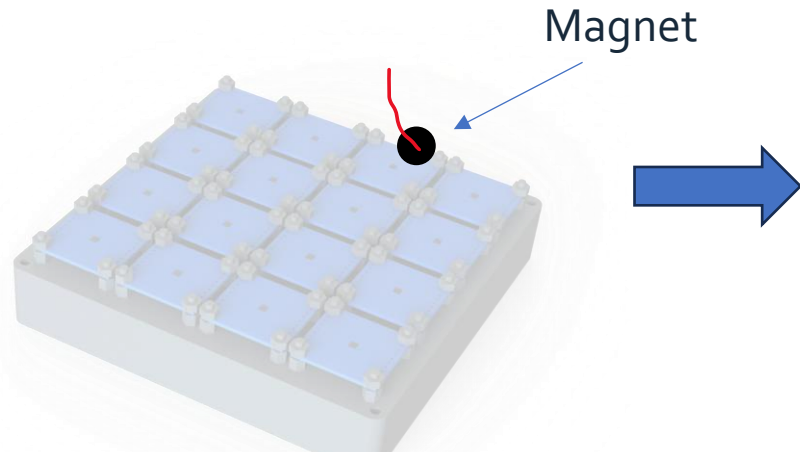


# Projectual Work – Warmup Start

*Real 3D Trajectory*

*3D representation in Python*

*2D representation in Python*

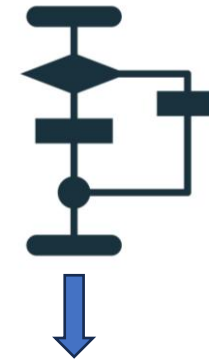


*Create the Behavior of Virtual Switches*



*A suggested layout follows the one of joypads, but you can be creative.*

*Partition the projected space into multiple switches.*



*Create Logic to detect pressed virtual switches*

"Pressed 3"

# Projectual Work – Your Task

1. Implement a Python pipeline for real-time visualization of the recorded 3D trajectory of a marker position. **Since it's real-time, optimization in updating the charts is required!**  
**Reminder:** marker position is represented by its coordinates: x, y, z (ignore mx, my, mz)
2. Before visualization, you must **flip the sign of the z-coordinate** in the code.  
The visualization should include:
  - A 3D trajectory plot with the following features
    - Fixed x-axis limits: [-0.06, 0.06] m
    - Fixed y-axis limits: [-0.06, 0.06] m
    - Fixed z-axis limits: [-0.03, 0.09] m
  - A parallel 2D trajectory plot (projection on x-y plane) with the following features
    - Fixed x-axis limits: [-0.06, 0.06] m
    - Fixed y-axis limits: [-0.06, 0.06] m
3. To preserve motion history, you should store the marker coordinates in an array containing a fixed number of previous samples. This stored trajectory should be updated as new marker positions are received. **Try testing different memory lengths and evaluate which value offers the best** trade-off among smooth visualization, clear trajectory history, responsiveness, and computational efficiency.

# Projectual Work – Your Task

4. From the 2D trajectory visualization, you should **extract a pixel-based 2D representation of the marker motion**. The goal is to convert the recorded 2D trajectory into an image-like format that can later be used as input for a neural network for digit recognition.
- The 2D trajectory should be mapped onto a fixed-size pixel grid, for example:  $28 \times 28$  pixels or  $64 \times 64$  pixels
- Use the fixed 2D axis limits: x:  $[-0.06, 0.06]$  m, y:  $[-0.06, 0.06]$  m
  - Each marker position should be converted from physical coordinates into pixel coordinates on the grid. The resulting image should represent the path drawn by the marker in 2D.
  - When developing the code, consider:
    - The chosen image resolution.
    - How to map physical coordinates to pixels.
    - Whether the trajectory should be represented as binary pixels or grayscale intensity.
    - Whether older trajectory points should fade over time.
    - How to store the generated images for later neural network training and testing (next class).



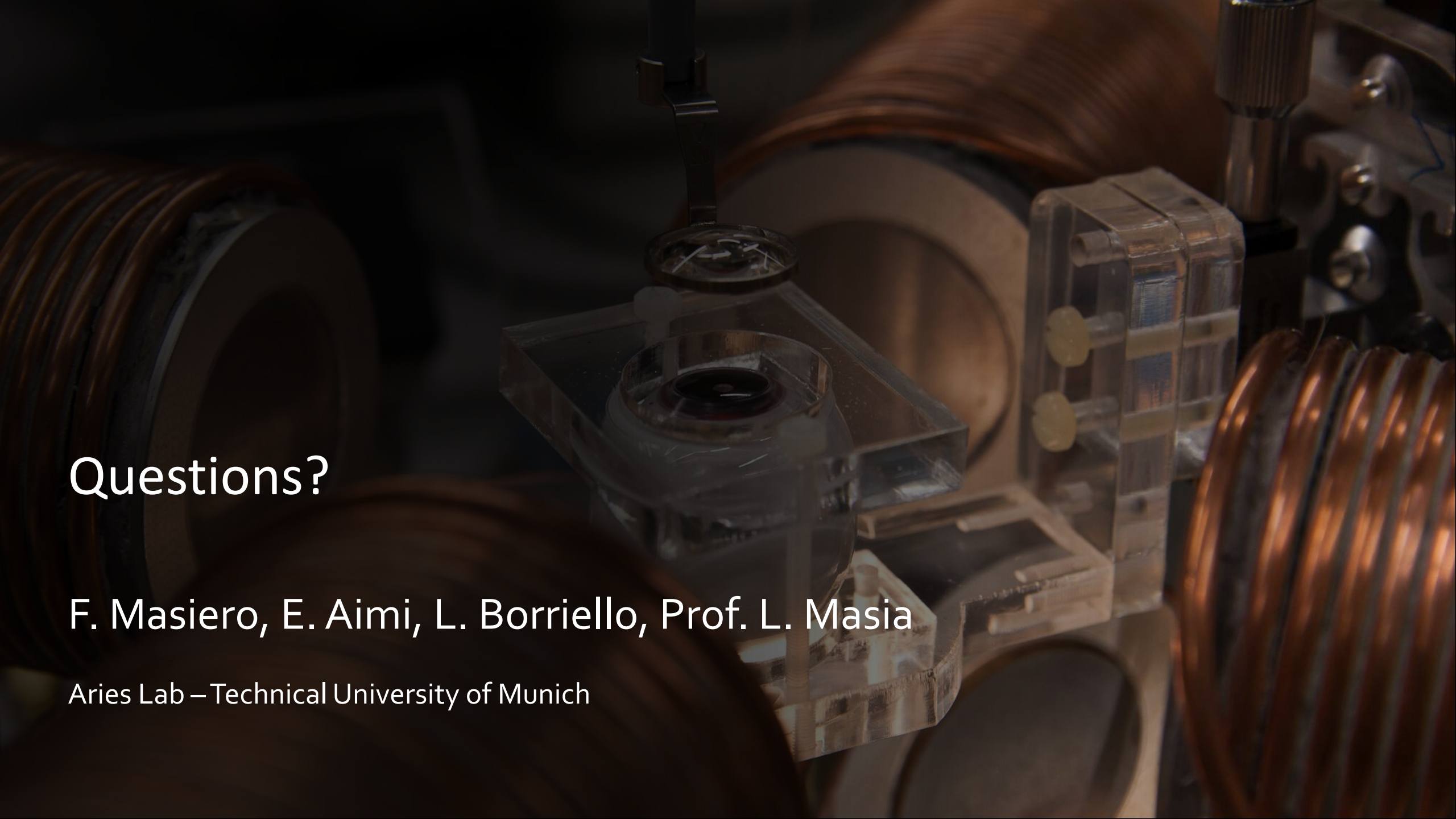
# Projectual Work – Your Task

5. Develop a separate 2D visualization that overlays a geometric rendering of virtual switch buttons, inspired by joystick/D-pad layouts such as the attached example.
  - The rendering must use geometric primitives (for example, lines, polygons, circles, or patches).
  - The 2D visualization should include:
    - the virtual switches drawn in fixed positions,
    - the cursor corresponding to the current marker trajectory,
    - and the recent trajectory, if useful.
  - These layouts can be explored:
    - Directional buttons: up, down, left, right
    - Numbered buttons: for example, 1, 2, 3, 4
    - All of them together (4 of the left and 4 on the right)
  - Implement logic to determine if and which switch is being pressed.
  - The result should be printed in the terminal, for example: "switch 3 pressed", "left arrow pressed". Define clear geometric regions for each switch and determine how to robustly detect a press from the marker position. They should also consider how to avoid ambiguous detections when the cursor is close to the boundary between two buttons.

# Projectual Work – Common Practice

## THE HOLY RULES

- ☐ Do you think your solution is the best? Maybe not, discuss it with the others and with us; try to improve it!
- ☐ Keep your code tidy and well documented, not only in all the scripts you produce, but also in the repo Readme.MD
- ☐ Keep the repo clean and updated.



Questions?

F. Masiero, E. Aimi, L. Borriello, Prof. L. Masia

Aries Lab – Technical University of Munich